

Factory Pattern

The Factory Pattern is a **creational design pattern** that provides an interface for creating objects but allows subclasses to alter the type of objects that will be created.

In simpler terms:

Rather than calling a constructor directly to create an object, we use a factory method to create that object based on some input or condition.

When Should You Use It?

We can use the Factory Pattern when:

- The client code needs to work with multiple types of objects.
- The decision of which class to instantiate must be made at runtime.
- The instantiation process is complex or needs to be controlled.

Real-World Analogy: Ordering Pizza

Imagine you walk into a pizza shop and say, "I'd like a pizza." The shop doesn't ask you to go into the kitchen and make it yourself. Instead, it asks, "Which type? Margherita? Pepperoni? Veggie?" Based on your choice, the kitchen (factory) creates the specific pizza for you and hands it over.

You (the client), don't care **how** it's made or what specific class of ingredients is used. You just want your pizza. The **factory (kitchen)** handles the creation logic behind the scenes.

This is exactly what the Factory Pattern does in code: it creates an object based on some input without exposing the instantiation logic to the client.

Basic Structure of Factory Pattern

The Factory Pattern typically consists of the following components:

- **Product:** It is an *interface* or *abstract class* that defines the methods the product must implement.
- **Concrete Products:** The *concrete classes* that implement the Product interface.
- **Factory:** A class with a method that returns different concrete products based on input.

Consider the following example code snippet:

```
#include <iostream>

#include <string>

using namespace std;

// Interface

class Shape {

public:

    virtual void draw() = 0; // Pure virtual function

    virtual ~Shape() {}

};

// Class implementing the Shape Interface
```

```
class Circle : public Shape {  
  
public:  
  
    void draw() override {  
  
        cout << "Drawing Circle" << endl;  
  
    }  
  
};
```

// Class implementing the Shape Interface

```
class Square : public Shape {  
  
public:  
  
    void draw() override {  
  
        cout << "Drawing Square" << endl;  
  
    }  
  
};
```

// Factory Class

```
class ShapeFactory {  
  
public:  
  
    /* Method that takes the type of shape as input  
    and returns the corresponding object */  
  
    Shape* getShape(const string& shapeType) {
```

```
    if (shapeType == "CIRCLE") {  
        return new Circle();  
    } else if (shapeType == "SQUARE") {  
        return new Square();  
    }  
    return nullptr;  
}  
};
```

```
// Driver code
```

```
int main() {  
    // Object of ShapeFactory is initialized  
    ShapeFactory shapeFactory;  
  
    // Get a Circle object and call its draw method  
    Shape* shape1 = shapeFactory.getShape("CIRCLE");  
    if (shape1) shape1->draw();  
  
    // Get a Square object and call its draw method  
    Shape* shape2 = shapeFactory.getShape("SQUARE");  
    if (shape2) shape2->draw();  
}
```

```
// Clean up dynamic memory

delete shape1;

delete shape2;


return 0;
}


// Interface

interface Shape {

    void draw();

}


// Class implementing the Shape Interface

class Circle implements Shape {

    @Override

    public void draw() {

        System.out.println("Drawing Circle");

    }

}
```

```
// Class implementing the Shape Interface
```

```
class Square implements Shape {
```

```
    @Override
```

```
    public void draw() {
```

```
        System.out.println("Drawing Square");
```

```
    }
```

```
}
```

```
// Factory Class
```

```
class ShapeFactory {
```

```
    /* Method that takes the type of shape as input
```

```
    and returns the cirresponding object */
```

```
    public Shape getShape(String shapeType) {
```

```
        if (shapeType.equalsIgnoreCase("CIRCLE")) {
```

```
            return new Circle();
```

```
        } else if (shapeType.equalsIgnoreCase("SQUARE")) {
```

```
            return new Square();
```

```
        }
```

```
        return null;
```

```
    }
```

```
}
```

```
// Driver code
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        // Object of ShapeFactory is initialized
```

```
        ShapeFactory shapeFactory = new ShapeFactory();
```

```
        // Get a Circle object and call its draw method
```

```
        Shape shape1 = shapeFactory.getShape("CIRCLE");
```

```
        shape1.draw();
```

```
        // Get a Square object and call its draw method
```

```
        Shape shape2 = shapeFactory.getShape("SQUARE");
```

```
        shape2.draw();
```

```
    }
```

```
}
```

```
# Interface
```

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
@abstractmethod
```

```
def draw(self):
```

```
    pass
```

```
# Class implementing the Shape Interface
```

```
class Circle(Shape):
```

```
    def draw(self):
```

```
        print("Drawing Circle")
```

```
# Class implementing the Shape Interface
```

```
class Square(Shape):
```

```
    def draw(self):
```

```
        print("Drawing Square")
```

```
# Factory Class
```

```
class ShapeFactory:
```

```
    # Method that takes the type of shape as input
```

```
    # and returns the corresponding object
```

```
    def get_shape(self, shape_type):
```

```
        if shape_type.lower() == "circle":
```

```
            return Circle()
```

```
elif shape_type.lower() == "square":  
    return Square()  
  
return None  
  
# Driver code  
  
if __name__ == "__main__":  
    # Object of ShapeFactory is initialized  
    shape_factory = ShapeFactory()  
  
    # Get a Circle object and call its draw method  
    shape1 = shape_factory.get_shape("CIRCLE")  
    if shape1: shape1.draw()  
  
    # Get a Square object and call its draw method  
    shape2 = shape_factory.get_shape("SQUARE")  
    if shape2: shape2.draw()
```

Here, `ShapeFactory` is the factory that returns different objects (`Circle`, `Square`) based on input.

Real-life Product Example – Logistics Services

Let's say you are building a logistics application that needs to handle different types of transport services: **By Road**, **By Air**, etc.

Bad Practice: Not Following Factory Pattern

Consider the following code snippet where object creation logic is tightly coupled with business logic:

```
#include <iostream>

#include <string>

using namespace std;

// Logistics Interface

class Logistics {

public:

    virtual void send() = 0; // Pure virtual function

    virtual ~Logistics() {}

};

// Class implementing the Logistics Interface

class Road : public Logistics {

public:

    void send() override {
```

```
        cout << "Sending by road logic" << endl;
    }
};

// Class implementing the Logistics Interface
class Air : public Logistics {
public:
    void send() override {
        cout << "Sending by air logic" << endl;
    }
};
```

```
// Class implementing Logistics Service
class LogisticsService {
public:
    void send(const string& mode) {
        if (mode == "Air") {
            Logistics* logistics = new Air();
            logistics->send();
            delete logistics;
        } else if (mode == "Road") {
```

```
        Logistics* logistics = new Road();

        logistics->send();

        delete logistics;

    }

}
```

// Driver code

```
int main() {

    LogisticsService service;

    service.send("Air");

    service.send("Road");

    return 0;

}
```

// Logistics Interface

```
interface Logistics {

    void send();

}
```

```
// Class implementing the Logistics Interface

class Road implements Logistics {

    @Override

    public void send() {

        System.out.println("Sending by road logic");

    }

}
```

```
// Class implementing the Logistics Interface

class Air implements Logistics {

    @Override

    public void send() {

        System.out.println("Sending by air logic");

    }

}
```

```
// Class implementing Logistics Service

class LogisticsService {

    public void send(String mode) {

        if (mode.equals("Air")) {

            Logistics logistics = new Air();

        }

    }

}
```

```
        logistics.send();  
    } else if (mode.equals("Road")) {  
        Logistics logistics = new Road();  
        logistics.send();  
    }  
}  
}
```

// Driver code

```
class Main {  
    public static void main(String[] args) {  
        LogisticsService service = new LogisticsService();  
        service.send("Air");  
        service.send("Road");  
    }  
}
```

Logistics Interface

from abc import ABC, abstractmethod

```
class Logistics(ABC):
```

```
@abstractmethod
```

```
def send(self):
```

```
    pass
```

```
# Class implementing the Logistics Interface
```

```
class Road(Logistics):
```

```
    def send(self):
```

```
        print("Sending by road logic")
```

```
# Class implementing the Logistics Interface
```

```
class Air(Logistics):
```

```
    def send(self):
```

```
        print("Sending by air logic")
```

```
# Class implementing Logistics Service
```

```
class LogisticsService:
```

```
    def send(self, mode):
```

```
        if mode.lower() == "air":
```

```
            logistics = Air()
```

```
            logistics.send()
```

```
        elif mode.lower() == "road":
```

```
logistics = Road()
```

```
logistics.send()
```

```
# Driver code
```

```
if __name__ == "__main__":
```

```
    service = LogisticsService()
```

```
    service.send("Air")
```

```
    service.send("Road")
```

Understanding the Issue:

In the LogisticsService class:

- The object of `Air` or `Road` is directly instantiated **based on string comparison**.
- The **object creation logic is embedded** inside the business logic (`send` method).
- This violates the **Open/Closed Principle** — if you want to add a new mode (e.g., `Ship`), you have to **modify** the `send` method.

Problems:

- **Tight Coupling:** LogisticsService depends directly on Air and Road classes.
- **Hard to Extend:** Adding a new mode (e.g., Drone, Ship) requires modifying existing code.
- **No Separation of Concerns:** Object creation and business logic are mixed.
- **Code Duplication:** Repeated instantiation and `send()` logic.

- **Testing & Maintenance Nightmare:** Hard to test independently or mock logistics.

Good Practice: Following Factory Pattern

Let's now apply the Factory Pattern to clean this up and make it scalable.

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Logistics Interface
```

```
class Logistics {
```

```
public:
```

```
    virtual void send() = 0; // Pure virtual function
```

```
    virtual ~Logistics() {}
```

```
};
```

```
// Class implementing the Logistics Interface
```

```
class Road : public Logistics {
```

```
public:
```

```
    void send() override {
```

```
        cout << "Sending by road logic" << endl;
```

```
    }
```

```
};
```

```
// Class implementing the Logistics Interface
```

```
class Air : public Logistics {
```

```
public:
```

```
    void send() override {
```

```
        cout << "Sending by air logic" << endl;
```

```
    }
```

```
};
```

```
// Factory Class taking care of Logistics
```

```
class LogisticsFactory {
```

```
public:
```

```
    static Logistics* getLogistics(const string& mode) {
```

```
        if (mode == "Air") {
```

```
            return new Air();
```

```
        } else if (mode == "Road") {
```

```
            return new Road();
```

```
        }
```

```
        throw invalid_argument("Unknown logistics mode: " + mode);
```

```
    }
```

```
};
```

```
// Class implementing the Logistics Services
```

```
class LogisticsService {
```

```
public:
```

```
    void send(const string& mode) {
```

```
        // Using the Logistics Factory to get the desired object based on the  
        mode
```

```
        Logistics* logistics = LogisticsFactory::getLogistics(mode);
```

```
        logistics->send();
```

```
        delete logistics; // Clean up memory
```

```
    }
```

```
};
```

```
// Driver Code
```

```
int main() {
```

```
    LogisticsService service;
```

```
    service.send("Air");
```

```
    service.send("Road");
```

```
    return 0;
```

```
}
```

```
// Logistic Interface
```

```
interface Logistics {
```

```
    void send();
```

```
}
```

```
// Class implementing the Logistics Interface
```

```
class Road implements Logistics {
```

```
    @Override
```

```
    public void send() {
```

```
        System.out.println("Sending by road logic");
```

```
    }
```

```
}
```

```
// Class implementing the Logistics Interface
```

```
class Air implements Logistics {
```

```
    @Override
```

```
    public void send() {
```

```
        System.out.println("Sending by air logic");
```

```
    }
```

```
}
```

```
// Factory Class taking care of Logistics
```

```
class LogisticsFactory {  
  
    public static Logistics getLogistics(String mode) {  
  
        if (mode.equalsIgnoreCase("Air")) {  
  
            return new Air();  
  
        } else if (mode.equalsIgnoreCase("Road")) {  
  
            return new Road();  
  
        }  
  
        throw new IllegalArgumentException("Unknown logistics mode: " +  
mode);  
  
    }  
}
```

```
// Class implementing the Logistics Services
```

```
class LogisticsService {  
  
    public void send(String mode) {  
  
        /* Using the Logistics Factory to get the  
        desired object based on the mode */  
  
        Logistics logistics = LogisticsFactory.getLogistics(mode);  
  
        logistics.send();  
  
    }  
}
```

```
}
```

```
// Driver Code
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        LogisticsService service = new LogisticsService();
```

```
        service.send("Air");
```

```
        service.send("Road");
```

```
    }
```

```
}
```

```
# Logistics Interface
```

```
from abc import ABC, abstractmethod
```

```
class Logistics(ABC):
```

```
    @abstractmethod
```

```
    def send(self):
```

```
        pass
```

```
# Class implementing the Logistics Interface
```

```
class Road(Logistics):
```

```
def send(self):  
  
    print("Sending by road logic")
```

Class implementing the Logistics Interface

```
class Air(Logistics):  
  
    def send(self):  
  
        print("Sending by air logic")
```

Factory Class taking care of Logistics

```
class LogisticsFactory:  
  
    @staticmethod  
  
    def get_logistics(mode):  
  
        if mode.lower() == "air":  
  
            return Air()  
  
        elif mode.lower() == "road":  
  
            return Road()  
  
        else:  
  
            raise ValueError(f"Unknown logistics mode: {mode}")
```

Class implementing the Logistics Services

```
class LogisticsService:
```

```
def send(self, mode):  
  
    # Using the Logistics Factory to get the desired object based on the  
    mode  
  
    logistics = LogisticsFactory.get_logistics(mode)  
  
    logistics.send()
```

Driver Code

```
if __name__ == "__main__":  
  
    service = LogisticsService()  
  
    service.send("Air")  
  
    service.send("Road")
```

Understanding the Improvement:

In this refactored code:

- The object creation logic is moved to the `LogisticsFactory`.
- The `LogisticsService` class now only focuses on business logic.
- Adding a new mode (e.g., Ship) only requires modifying the factory, not the service.

Benefits:

- **Loose Coupling:** The service is decoupled from specific logistics classes.
- **Open/Closed Principle:** New modes can be added without modifying existing code.
- **Separation of Concerns:** Object creation and business logic are separated.

- **No Code Duplication:** Instantiation logic is centralized in the factory.
 - **Easier Testing & Maintenance:** Each component can be tested independently.
-

Pros of Factory Pattern

- **Promotes Loose Coupling:**
 - The client code is decoupled from the actual instantiation of classes.
 - You work with interfaces rather than concrete classes.
 - **Enhances Extensibility (OCP – Open/Closed Principle):**
 - You can introduce new classes (e.g., new types of logistics like **Ship**) without modifying existing client code.
 - The system becomes easier to scale and extend.
 - **Centralizes Object Creation (SRP – Single Responsibility Principle):**
 - The responsibility of object creation is moved to a dedicated factory class.
 - Business logic stays clean and focused only on "what to do" with the object.
 - **Increases Flexibility:**
 - The decision of "which object to create" can be deferred to runtime based on input, config, or logic.
 - Makes your system adaptable to dynamic requirements.
 - **Improves Code Reusability:**
 - Common instantiation logic can be reused from a single factory.
 - Avoids code duplication when creating similar objects in different parts of the system.
-

Cons of Factory Pattern

- **Increased Complexity:** Introduces additional layers (factory classes/interfaces) which might be overkill for very small programs.
- **More Code Overhead:** Requires writing extra code like factory classes and interfaces, which might look unnecessary in simpler use-cases.

Class Diagram

